

목차 (Table of Contents)

모델 크기와 메모리

파라미터 수 → 메모리 변환

BF16 vs FP16 차이

양자화와 성능 손실

INT8 / INT4 양자화 시 BF16 대비 성능 영향

CUDA Unified Memory vs iGPU Zero-Copy

공통점

핵심 차이

Unified Memory가 느린 이유

Pinned Memory vs Pageable Memory

이론 0.6초 vs 실제 273초 격차의 원인

CUDA Unified Memory vs device_map="auto" 구분

RTSS 2022 Demand Layering이 iGPU 대상인 이유

HuggingFace 라이브러리

Transformers

Accelerate

관계

모듈 단위 스와핑 vs device_map="auto" vs 레이어 단위 스와핑

교수님 아이디어: 모듈 단위(stage-level) 동적 스와핑

device_map="auto"와의 차이

모듈 스와핑만으로는 12GB에 불충분한 이유

연구 방향 의사결정

배제 대상: 모델 정확도를 떨어뜨리는 방식

학습 정보 정리

작업 중 질의응답을 통해 습득한 지식과 정보를 정리하는 문서입니다. 마지막 업데이트:
2026-02-25

모델 크기와 메모리

파라미터 수 → 메모리 변환

- 파라미터 수(B) = 모델의 가중치 숫자 개수 (예: 11.08B = 110.8억 개)
- 메모리 = 파라미터 수 × 데이터 타입 바이트 크기

데이터 타입	바이트/파라미터	11.08B 모델 기준
FP32	4 bytes	~44.3 GB
BF16 / FP16	2 bytes	~22.2 GB
INT8	1 byte	~11.1 GB
INT4	0.5 bytes	~5.5 GB

BF16 vs FP16 차이

둘 다 16비트(2바이트)지만 비트 배분이 다름:

	FP16 (float16)	BF16 (bfloat16)
부호	1비트	1비트
지수(exponent)	5비트	8비트
가수(mantissa)	10비트	7비트
표현 범위	$\pm 65,504$	$\pm 3.4 \times 10^{38}$ (FP32와 동일)
정밀도	더 정밀	덜 정밀

- **FP16**: 정밀도 높지만 범위 좁음 → 큰 값에서 오버플로우 위험 (loss scaling 필요)
- **BF16**: 정밀도 낮지만 범위가 FP32와 동일 → 딥러닝에서 안정적
- BF16은 Google이 TPU용으로 설계, NVIDIA Ampere(RTX 30xx) 이후 GPU 지원
- **딥러닝에서 BF16 선호 이유**: 가중치/gradient의 정밀도는 소수점 10자리까지 불필요하지만, 값의 범위는 넓어야 함. FP16은 $\pm 65,504$ 초과 시 오버플로우(inf) 발생 → loss scaling 필요. BF16은 FP32와 동일 범위라 loss scaling 없이 안정적 동작. 정밀도 손실(10→7비트)은 모델 품질에 거의 영향 없음이 실험적으로 확인됨

양자화와 성능 손실

INT8 / INT4 양자화 시 BF16 대비 성능 영향

양자화	메모리 절감	정확도 손실 (일반 LLM)	비고
BF16 (기준)	0%	0%	기준선
INT8	50%	~0.1-1%	거의 무손실
INT4	75%	~1-5%	대부분 허용 범위
INT4 (나쁜 경우)	75%	~5-10%	소형 모델이나 특수 태스크

- 모델이 클수록(10B+) 양자화 손실에 강함, 소형(1-3B)은 손실 큼
- 추론 속도는 오히려 빨라짐 (메모리 대역폭 병목 해소, INT4가 BF16 대비 2-3x 빠름)
- 양자화 방법에 따라 차이: RTN(단순, 손실 큼) < GPTQ/AWQ(캘리브레이션, 손실 적음)
- 도메인 특화 태스크(자율주행 궤적 예측 등)는 일반 벤치마크와 품질 기준이 다를 수 있음

CUDA Unified Memory vs iGPU Zero-Copy

공통점

- CPU와 GPU가 같은 메모리 주소 사용, 명시적 복사 없이 데이터 접근

핵심 차이

	iGPU Zero-Copy	CUDA Unified Memory (dGPU)
물리 메모리	CPU/GPU 동일한 물리 RAM 공유	CPU RAM과 GPU VRAM 물리적 분리
데이터 이동	실제 복사 없음 (진짜 zero-copy)	페이지 폴트 시 PCIe로 마이그레이션
VRAM 초과 시	해당 없음 (RAM=VRAM)	자동 스와핑 (페이지 폴트 기반)
대표 HW	Jetson, Apple M시리즈, Intel iGPU	RTX/Quadro/A100 등 discrete GPU

Unified Memory가 느린 이유

1. 페이지 폴트 기반 — GPU 실행 중단 → 드라이버 트랩 → PCIe 전송 → 재개
2. 양방향 eviction — VRAM 포화 시 기존 페이지 내보내야 새 페이지 가져옴
3. 소규모 페이지 단위 전송 (4KB~2MB) → PCIe 대역폭 활용률 저하
4. 접근 패턴 예측 불가 → 프리페칭 불가

Pinned Memory vs Pageable Memory

- **Pageable**: OS가 디스크로 스왑 가능 → GPU DMA 전 임시 pinned 버퍼로 복사 필요 (추가 단계)
- **Pinned**: OS가 스왑 불가하도록 고정 → GPU DMA가 직접 접근, 비동기 전송 가능
- Pinned의 장점: 중간 복사 제거 + CPU 개입 없이 DMA 독립 전송 (GPU 연산과 동시 실행)
- Pinned의 단점: 시스템 RAM 고정 점유 → 과다 사용 시 OS 메모리 부족 위험
- WSL2에서 D2H: pageable 2.48 GB/s vs pinned 11.8 GB/s (4.8배 차이, 가상화 오버헤드)
- **현실 제약**: 시스템 RAM(15GB) < 모델(22GB)이므로 전체를 pinned로 올릴 수 없음 → 소규모 pinned 버퍼(1~2GB)를 할당하고 레이어를 번갈아 올리는 방식이 현실적 (On-Demand Layering + 스왑 최적화 결합)

이론 0.6초 vs 실제 273초 격차의 원인

- **이론**: 9.5GB를 한 덩어리로 연속 전송 가정 ($9.5\text{GB} \div 16\text{GB/s} \approx 0.6\text{초}$)
- **실제**: 토큰 생성마다 레이어 접근 → 페이지 폴트 → 소규모 전송 반복
- 반복 횟수: $256\text{토큰} \times 36\text{레이어} = 9,216\text{번} + \text{레이어 접근}$
- per-transfer 오버헤드: 0.36ms/건 (WSL2 측정)
- 양방향 경합: evict(내보내기) + fetch(가져오기) 동시 발생
- 연구 방향 1~3의 핵심: 페이지 폴트 기반 자동 스와핑 → 명시적/계획적 전송으로 대체

CUDA Unified Memory vs device_map="auto" 구분

- **CUDA Unified Memory**: GPU 드라이버 레벨의 자동 페이지 스와핑. `.to("cuda")` 로 VRAM 초과 할당 시 투명하게 동작. 베이스라인에서 12GB로 Alpamayo가 돌아간 이유.
- **device_map="auto"**: Accelerate 라이브러리가 레이어를 GPU/CPU에 미리 나눠 배치. Alpamayo에서는 커스텀 토큰 처리 비호환으로 추론 실패.
- 두 개는 완전 다른 메커니즘. 현재 동작하는 것은 Unified Memory이고, 그것이 느리기 때문에 최적화 연구 중.
- **device_map="auto"가 Alpamayo에서 실패하는 이유**: Alpamayo는 `fuse_traj_tokens()` 라는 커스텀 연산에서 VLM 출력 텐서와 trajectory 토큰 테이블을 `masked_scatter` 로 합침. device_map이 텐서를 GPU/CPU에 분산 배치하면 이 연산에서 디바이스 불일치 에러 발생. 해결하려면 소스코드 수정이 필요 → 커스텀 오프로딩 구현이 연구 핵심 과제.

RTSS 2022 Demand Layering이 iGPU 대상인 이유

- iGPU는 CPU RAM을 공유하므로 “CPU로 오프로딩” 불가
- 대신 NVMe SSD로 오프로딩하는 새로운 접근이 필요

HuggingFace 라이브러리

Transformers

- HuggingFace의 모델 로딩/추론 프레임워크
- `from_pretrained()` 로 모델 다운로드, config 파싱, 가중치 로딩 등 처리
- LLM/VLM 생태계의 사실상 표준

Accelerate

- HuggingFace의 분산/멀티 디바이스 실행 지원 라이브러리

- `device_map="auto"` 등 GPU/CPU 자동 배치 기능 제공
- 모델이 단일 GPU에 안 들어갈 때 여러 디바이스에 분배
- 배치 순서: GPU VRAM부터 채움 → 초과분은 CPU RAM → 그래도 부족하면 디스크
- 추론 시 CPU에 있는 레이어를 실행 차례에 GPU로 이동 → 완료 후 CPU로 반환 (자동)

관계

사용자 코드 → Transformers (모델 로딩/추론) → Accelerate (디바이스 배치)
→ PyTorch (GPU 연산)

모듈 단위 스와핑 vs `device_map="auto"` vs 레이어 단위 스와핑

교수님 아이디어: 모듈 단위(stage-level) 동적 스와핑

- 비전 인코더(1.15GB) 로드 → 실행 → 언로드 → VLM(15.17GB) 로드 → 실행 → 언로드 → Diffusion(4.56GB) 로드 → 실행 → 언로드
- Peak VRAM = $\max(1.15, 15.17, 4.56) = 15.17\text{GB}$ (전체 21GB 대비 감소)
- 한 번에 하나의 모듈만 VRAM에 존재

`device_map="auto"`와의 차이

	교수님 아이디어 (모듈 스와핑)	<code>device_map="auto"</code>	연구 방향 1&2 (레이어 + 모듈)
스와핑 단위	모듈(stage)	레이어	레이어 + 모듈
배치 시점	동적 (사용 시 로드/언로드)	정적 배치 + 동적 이동	동적 + 파이프라이닝
Peak VRAM	~15.17GB	설정에 따라 다름	12GB 목표
Alpamayo 호환	커스텀 구현 필요	X (fuse_traj_tokens 실패)	커스텀 구현 필요

모듈 스와핑만으로는 12GB에 불충분한 이유

- VLM 단독이 15.17GB → 12GB VRAM 초과
- 결국 VLM 내부에서도 레이어 단위 스와핑이 추가로 필요

- 이것이 연구 방향 1(On-Demand Layering)과 방향 2(Memory Pipelining)에서 탐색한 내용
- 최종적으로 모듈 단위 + 레이어 단위 복합 스와핑이 해법

연구 방향 의사결정

배제 대상: 모델 정확도를 떨어뜨리는 방식

- 양자화 (INT8, INT4, 하이브리드 양자화 등)
- 레이어 가지치기 (layer pruning)
- 동적 해상도 축소
- 모델 구조 변경 전반

→ 연구 초점: 모델 자체는 그대로 두고, 시스템/메모리 레벨에서 최적화 (On-Demand Layering, 메모리 파이프라이닝, CPU-GPU 스왑 최적화 등)
